Evidence-Closed Knowledge Maintenance for LLM Agents: Design and Empirical Evaluation of Agent Wiki

Zijun Chu[a,*]

[a]*Independent Researcher, Room 1303, No. 19, Lane 899, Xinfeng Middle Road, Qingpu District, Shanghai, China*

Abstract

Context: Large language model (LLM) agents need durable external knowledge, but file-based corpora provide little assurance that source evidence remains reachable after agents synthesize or reorganize it. **Objective:** This study investigates whether an executable evidence-closure invariant can make local Markdown knowledge maintenance structurally auditable at documentation scale. **Methods:** We designed and implemented Agent Wiki, a local-first system that separates raw evidence from synthesized wiki pages and checks four closure conditions for processed sources. We evaluated release v0.2.1 using a public FlexSim documentation-derived graph containing 1,092 raw records, 88 wiki records, and 5,163 edges. We then injected 120 faults from four maintenance fault classes into a 1,000-record fixture and compared Agent Wiki with an existence-only link baseline. Finally, we measured end-to-end scan and closure checking over synthetic vaults of 100–2,000 raw records, with ten repetitions per size. **Results:** All 1,092 public raw records retained a source URL, a SHA-256 content hash, a resolved related wiki target, and a reciprocal wiki-to-evidence backlink. Agent Wiki detected all 120 injected faults with no false positives; the existence-only baseline detected 30. Median end-to-end checking time ranged from 56.8 ms for 100 raw records to 1,302.3 ms for 2,000. **Conclusion:** Executable evidence closure detects lifecycle failures beyond broken links while retaining low local checking cost at the studied scale. The results establish structural integrity, not semantic correctness or superior answer quality relative to retrieval-augmented generation.

*Corresponding author.

Email address: chunimav5@gmail.com (Zijun Chu)

URL: https://github.com/NimaChu/agent-wiki (Zijun Chu)

Keywords: LLM agents, knowledge maintenance, traceability, provenance, local-first software, Markdown, empirical software engineering

1. Introduction

Large language model (LLM) agents increasingly perform multi-step software and knowledge-work tasks. They inspect repositories, read technical documentation, call external tools, modify files, and resume work across sessions. Systems and benchmarks such as ReAct, SWE-agent, and SWE-bench demonstrate the value of combining language models with actions and software environments [1, 2, 3]. These workflows also expose a less visible engineering problem: the knowledge accumulated by an agent must remain inspectable and maintainable after the immediate conversation ends.

External memory is commonly implemented through chat history, application memory, document retrieval, vector indexes, or human-authored notes. Retrieval- augmented generation (RAG) and graph-based retrieval improve access to external information [4, 5]. Memory-oriented systems such as MemGPT manage information across context tiers [6]. These approaches address what information should be presented to a model, but they do not by themselves define when a captured source has been responsibly integrated into a durable knowledge base. A source may be marked processed even though its synthesis page is missing, its target link is broken, the synthesis does not point back to evidence, or an explicit follow-up remains open.

This distinction is a software-quality concern. For a corpus maintained by humans and agents, structural consistency must be checkable independently of the model that created the notes. Plain files provide ownership and inspectability, but file ownership alone does not provide lifecycle invariants. A conventional broken-link checker can identify a reference to a missing file; it cannot determine that a processed source declares no durable target, that a wiki page fails to preserve a backlink to evidence, or that processing was closed while a follow-up flag remained active.

We address this problem with *evidence closure*: an executable invariant over a two-layer Markdown knowledge base. Raw notes preserve evidence and capture metadata; wiki pages contain durable synthesis. A raw source may be treated as processed only when it declares at least one related wiki target, those targets resolve, at least one related wiki page links back to the raw source, and no explicit follow-up remains. Agent Wiki implements this invariant as part of a local command-line workflow and exposes the same derived graph to search, maintenance queues, and an optional read-only dashboard.

The article makes four contributions:

1. an operational evidence-closure model for agent-maintained knowledge, expressed as conditions that can be checked over ordinary Markdown files;
2. an open-source implementation that separates evidence, synthesis, and derived visualization while requiring no hosted database or embedding service;
3. a reproducible empirical evaluation combining a public documentation-derived graph, seeded maintenance faults, and an existence-only baseline; and
4. a runtime study over controlled vaults containing up to 2,000 raw records, together with scripts and machine-readable results.

The evaluation is intentionally scoped to structural integrity. It does not claim that evidence closure proves factual correctness, improves user productivity, or outperforms RAG on answer quality. Instead, it asks whether a specific, reviewable maintenance property can be enforced and measured.

2. Background and Related Work

2.1. LLM Agent Memory and Retrieval

RAG combines parametric generation with non-parametric document retrieval and has become a standard architecture for knowledge-intensive language tasks [4]. GraphRAG constructs entity graphs and community summaries to support global questions over large corpora [5]. MemGPT treats context management as a tiered memory problem, moving information between constrained and external storage [6]. These systems show that external knowledge is central to useful long-running agents.

Agent Wiki operates at a different layer. It does not prescribe an answer-time retriever, chunking strategy, embedding model, or prompt. It maintains the inspectable corpus from which a retriever may later be built. This separation allows vector, lexical, graph-guided, or manual retrieval to share the same source and synthesis artifacts. It also allows integrity checks to run without an LLM or network service.

2.2. Local-First and Personal Knowledge Systems

Local-first software emphasizes ownership, offline availability, and user control over data [7]. Markdown knowledge tools such as Obsidian and Logseq demonstrate the practical value of file-native notes, backlinks, and graph views [12, 13]. Evergreen-note practices similarly encourage small, durable notes that evolve through linking and revision [11].

These tools make knowledge inspectable, but their generality leaves lifecycle policy to the user. Agent Wiki adds an agent-operating protocol: repository rules define which files are raw evidence, which files are synthesis, which metadata fields describe state, and which checks must pass before a source is considered processed. The graph remains derived from the files rather than becoming a second source of truth.

2.3. Traceability and Provenance

Traceability research has long treated links between artifacts as essential for understanding origins, dependencies, and change [8]. The W3C PROV family provides a general model for representing entities, activities, agents, and provenance relations [9]. Knowledge graphs provide a broader framework for representing entities and relations in a machine-processable graph [10].

Evidence closure is narrower than general provenance modeling. It does not attempt to encode every transformation or claim. It defines a minimum bidirectional traceability gate between a captured source and at least one durable synthesis artifact. The small scope makes the invariant easy to run in local development and continuous integration. It also exposes a concrete failure model that can be evaluated through mutation.

2.4. Research Gap

Prior work provides retrieval, memory management, local ownership, or general provenance models. The missing practical element is a lightweight, executable definition of *processed* for an agent-maintained file corpus. Table 1 summarizes this positioning. The categories are architectural tendencies rather than claims about every implementation.

Approach	Local files	Retrieval	Raw/synthesis split	Closure gate
Chat transcript	Sometimes	Context search	No	No
Vector/RAG pipeline	Optional	Yes	Optional	No
Markdown note system	Yes	Optional	User-defined	User-defined
Agent Wiki	Yes	Optional	Yes	Yes

Table 1: Architectural positioning of Agent Wiki.

3. Evidence-Closure Model

3.1. Two-Layer Knowledge Representation

Let R be the set of raw source notes and W the set of synthesized wiki pages. A raw note records capture metadata, including source URL, capture time, content hash, source quality, status, and intended wiki targets. Its body may contain captured text, extracted claims, image inventories, and processing notes. A wiki page contains one durable knowledge unit, such as a concept, method, API, workflow, entity, or comparison, and cites one or more raw notes.

Separating R and W serves two purposes. First, synthesis can change without rewriting captured evidence. Second, a reader can distinguish what a source contained from what an agent or human concluded from it. A single source may support multiple wiki pages, and a wiki page may synthesize multiple sources.

3.2. Closure Invariant

For a raw source $r \in R$, let $\text{related}(r)$ be its declared wiki targets, $\text{resolve}(t)$ resolve a target to a page or failure $\perp$, and $\text{links}(w)$ be the links emitted by wiki page w. The source is evidence-closed only if:

$$\begin{aligned} \text{closed}(r) \equiv\ & |\text{related}(r)| > 0 \\ & \wedge\ \forall t \in \text{related}(r),\ \text{resolve}(t) \neq \perp \\ & \wedge\ \exists t \in \text{related}(r) : r \in \text{links}(\text{resolve}(t)) \\ & \wedge\ \neg\text{needsFollowup}(r). \end{aligned} \tag{1}$$

The lifecycle rule is then:

$$\text{status}(r) = \texttt{processed} \ \Rightarrow\ \text{closed}(r). \tag{2}$$

Violations are reported with four reason codes: `missing-related`, `unresolved-related`, `missing-wiki-backlink`, and `explicit-followup`. These codes form the fault model used in the evaluation.

3.3. Scope of the Invariant

Closure is necessary but not sufficient for semantic quality. A backlink can exist while a synthesis is inaccurate. A content hash records a captured state but does not prove the trustworthiness of the source. A resolved target may be too broad or too narrow. Agent Wiki therefore reports closure as a mechanical property and retains source-quality metadata for human or agent judgment. Claim-level entailment, answer faithfulness, and source authority are outside Equations 1–2.

4. System Design and Implementation

4.1. Architecture

Agent Wiki is a self-contained repository implemented with Node.js scripts and an optional Vite/React dashboard. Figure 1 shows the data flow. Markdown remains the source of truth; graph JSON and the dashboard are rebuildable views.

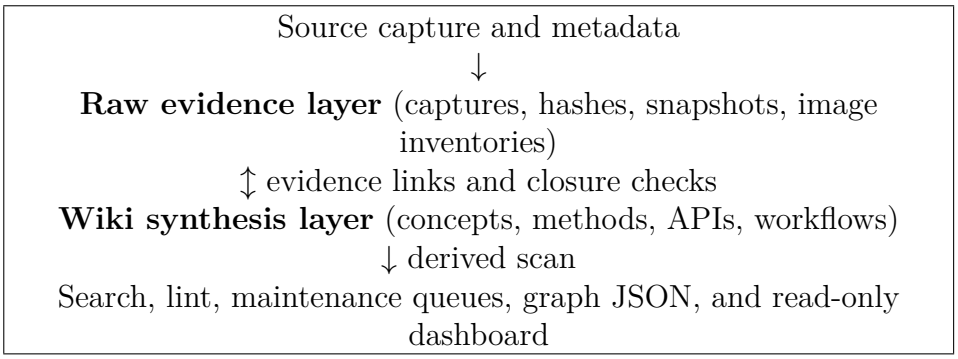

Figure 1: Agent Wiki keeps evidence and synthesis in Markdown and derives operational views from the same scanner.

The main directories are `raw/` for evidence, `wiki/` for synthesis, `templates/` for reusable schemas, `src/` for command-line logic, and `tools/wiki-dashboard/` for visualization. Public software and private knowledge can be separated through version-control policy.

4.2. Shared Scanner and Resolver

The scanner walks Markdown files, parses a deliberately constrained frontmatter subset, extracts body and metadata wiki links, and resolves targets by identifier, title, basename, or alias. It then emits nodes, edges, unresolved links, typed relations, incoming and outgoing degree counts, and excerpts. The same resolver supports status, lint, search, link repair, and dashboard graph generation. Reusing one resolver prevents each interface from applying a different definition of a valid link.

4.3. Operational Workflows

The command-line interface exposes capture, search, status, lint, garden review, link repair, image indexing, and graph generation through root npm scripts. The dashboard is read-only: write operations remain explicit Markdown edits or command-line actions. This choice makes changes visible to ordinary diff and review tools.

Image-rich sources receive a parallel evidence inventory. The image workflow extracts Markdown and HTML image references, resolves source-relative URLs, mirrors permitted images locally, and writes a machine-readable index. Images are not part of the closure invariant evaluated here, but their metadata preserves another evidence channel that text-only capture would lose.

4.4. Implementation of Closure Checks

After link resolution, the checker selects raw nodes whose status is `processed`. It verifies the four conditions in Equation 1. The check is deterministic and requires no model inference. Reports are emitted as JSON so failures can be inspected by humans, agents, or continuous-integration jobs. The current implementation favors portability and reviewability over a complete YAML or Markdown dialect.

5. Research Method

5.1. Research Questions

The evaluation addresses three research questions:

RQ1: To what extent does a documentation-scale public case study satisfy the observable evidence-closure preconditions?

RQ2: How effectively do evidence-closure checks detect seeded maintenance faults compared with an existence-only link baseline?

RQ3: What is the end-to-end runtime cost of scanning and closure checking as the number of Markdown records increases?

5.2. Artifact and Reproduction Package

The study evaluates Agent Wiki release `v0.2.1`. The software is licensed under the MIT License. Release `v0.2.1` and the public dataset are archived with the current DOI. The experiment script, JSON results, and CSV tables accompany this manuscript and are intended for the next archival release before submission. The complete experiment is invoked from the repository root with:

```
npm run paper:ist-evaluate
```

The script reads the public dataset without modifying it, constructs controlled fixtures in an operating-system temporary directory, runs the measurements, writes machine-readable results beside the manuscript, and deletes temporary fixtures.

5.3. RQ1: Public Case-Study Dataset

The public case study is derived from the English Autodesk FlexSim 2026 online help captured for an authorized local knowledge base. To avoid redistributing third-party documentation, the public dataset contains only whitelisted source metadata, structural features, hashes, and graph edges. It excludes captured text, webpage snapshots, screenshots, images, binaries, and local absolute paths.

The dataset contains `raw-sources.jsonl`, `wiki-pages.jsonl`, `edges.jsonl`, a schema, and aggregate metrics. For each raw record, the evaluation tests whether a source URL and 64-character SHA-256 hash are present, whether at least one declared wiki target resolved, and whether at least one resolved target has a reverse edge to the raw record. The last measure is the public-data counterpart of reciprocal evidence closure.

5.4. RQ2: Fault-Injection Protocol

We generated a clean fixture with 1,000 processed raw notes and 40 synthesized wiki pages. Each raw note mapped to one wiki page, and each wiki page linked back to its evidence. We then injected 30 non-overlapping instances of each fault class in Table 2, for 120 total faults. Fault locations were fixed by the script, making the run deterministic.

Fault	Mutation
`missing-related`	Remove the raw note's declared wiki target.
`unresolved-related`	Replace the target with a nonexistent wiki page.
`missing-wiki-backlink`	Remove the raw-source link from the related wiki page.
`explicit-followup`	Set `needs_followup` to true on a processed source.

Table 2: Seeded maintenance fault model.

The comparison baseline represents a conventional existence-only Markdown link checker: a fault is detected when an explicit link target does not resolve. It does not interpret source status, require a target declaration, inspect reverse evidence links, or enforce follow-up state. This baseline is intentionally narrow; it isolates the additional behavior contributed by evidence closure and is not a comparison with RAG or a database-backed knowledge system.

We report true positives, false positives, false negatives, precision, and recall against the seeded fault labels. Because the experiment is deterministic and exhaustively labels the constructed fixture, we use descriptive measures rather than inferential tests.

5.5. RQ3: Runtime Protocol

We generated clean fixtures containing 100, 500, 1,000, and 2,000 raw records. One wiki page was created for each group of at most 25 raw records, giving 4, 20, 40, and 80 wiki records. Every raw note was processed and evidence-closed. The measured operation includes filesystem traversal, UTF-8 reads, frontmatter and link parsing, link resolution, graph statistics, and processed-source closure checking.

For each size, two warm-up runs preceded ten measured repetitions. We report median, 95th percentile, minimum, and maximum wall-clock time. Measurements ran on Windows 11 with Node.js 20.20.0, an Intel Core i9-14900K, 32 logical CPUs, and 127.7 GiB of memory. The implementation is single-process for this workflow; the hardware description supports replication but is not used to claim general performance across machines.

6. Results

6.1. RQ1: Observable Closure at Documentation Scale

Table 3 reports the public case-study graph. All 1,092 raw records contained a source URL and SHA-256 content hash. Every raw record had at least one resolved related wiki target, and every raw record had a reciprocal edge from at least one related wiki page. Thus, all observable closure preconditions represented in the public dataset had 100% coverage.

Measure	Value
Raw source records	1,092
Synthesized wiki records	88
Graph nodes	1,180
Graph edges	5,163
Raw-to-wiki edges	2,273
Wiki-to-raw edges	2,310
Wiki-to-wiki edges	580
Raw records with source URL	1,092 (100%)
Raw records with SHA-256 hash	1,092 (100%)
Raw records with resolved related target	1,092 (100%)
Raw records with reciprocal backlink	1,092 (100%)

Table 3: Public FlexSim case-study dataset, release v0.2.1.

This result answers RQ1 for the released dataset. It demonstrates measurable traceability coverage; it does not establish that the 88 wiki pages are semantically complete or that every sentence is entailed by its sources.

6.2. RQ2: Detection of Seeded Maintenance Faults

Agent Wiki detected all 120 injected faults and emitted no additional fault labels, yielding precision and recall of 1.00 in the constructed fixture. The existence-only baseline detected the 30 unresolved targets but missed the 90 faults that did not contain a broken explicit link, yielding precision 1.00 and recall 0.25. Table 4 gives class-level results.

Fault class	Injected	Agent Wiki	Existence-only
Missing related target	30	30	0
Unresolved related target	30	30	30
Missing wiki backlink	30	30	0
Explicit follow-up	30	30	0
Total	120	120	30
Recall	—	1.00	0.25

Table 4: Fault-injection detection results. No false positives were observed.

The result answers RQ2 within the defined fault model: lifecycle-aware checks identify three classes of structural maintenance failure that link existence alone cannot represent. The perfect score is expected for checks designed against these explicit invariants and should be interpreted as mutation adequacy, not as an estimate of defect detection in arbitrary knowledge bases.

6.3. RQ3: Runtime Cost

Table 5 reports end-to-end timing. The median increased from 56.8 ms for 104 total nodes to 1,302.3 ms for 2,080 nodes. The 95th percentile at the largest size was 1,344.9 ms. No closure issues were reported in the clean fixtures.

Raw	Wiki	Nodes	Edges	Median (ms)	P95 (ms)
100	4	104	200	56.8	84.2
500	20	520	1,000	314.4	342.0
1,000	40	1,040	2,000	651.3	912.3
2,000	80	2,080	4,000	1,302.3	1,344.9

Table 5: End-to-end scan and closure-check runtime over ten repetitions.

These measurements answer RQ3 for the tested implementation and environment. At the studied documentation scale, complete local checking

remained below two seconds at the 95th percentile. The data suggest approximately proportional growth over this range, but the study does not fit or claim an asymptotic model.

7. Discussion

7.1. Evidence Closure as a Software Quality Gate

The evaluation shows why `processed` should be treated as a verifiable state rather than a descriptive label. Three quarters of the seeded faults did not require a broken link: the source could omit its durable target, the target could omit evidence, or a follow-up could remain open. These failures are visible only when lifecycle meaning is added to the graph.

The gate is deliberately small. This favors adoption because it can run in a local terminal, a coding-agent workflow, or continuous integration without a model call. It also makes failures actionable: each reason code points to a specific edit. More ambitious semantic checks can be layered on top, but they need not replace the deterministic foundation.

7.2. Relationship to Retrieval Systems

Agent Wiki should not be interpreted as a competitor to RAG. RAG decides what evidence to retrieve for a query; evidence closure decides whether the corpus maintains a minimum structural path between source and synthesis. A retrieval index can be generated from raw notes, wiki pages, or both. Keeping the file corpus authoritative makes the index disposable and rebuildable.

The present study therefore avoids an answer-quality comparison with a vector baseline. Such a comparison would require a query set, relevance judgments, model and embedding controls, and answer-faithfulness evaluation. Those are important next steps, but including them without an adequately labeled task set would weaken rather than strengthen the claim made here.

7.3. Practical Implications

For independent developers and small teams, the results support three practical uses. First, source-grounded knowledge can remain in ordinary files without giving up automated integrity checks. Second, agents can receive repository- local operating rules and commands instead of relying on hidden product memory. Third, public software can be separated from private corpora while derived metadata and evaluation fixtures remain reproducible.

The runtime result also makes closure suitable as a routine pre-commit or maintenance check at the studied scale. Larger repositories may require indexed incremental checking. The current shared scanner reads all Markdown files on each run, prioritizing simplicity over incremental performance.

8. Threats to Validity

Construct validity.. The dependent variables measure structural traceability, not knowledge correctness, completeness, usefulness, or user trust. Source URLs and hashes do not establish source authority. Reciprocal links do not establish entailment. We restrict conclusions accordingly.

Internal validity.. The four fault classes correspond directly to implemented closure conditions, so perfect Agent Wiki recall is not surprising. The purpose is to validate that the executable checks realize the stated invariant. The existence-only baseline is intentionally minimal and should not be generalized to mature knowledge-management products. Fault locations are non-overlapping and do not test interacting mutations.

External validity.. The observational dataset covers one technical-documentation domain. The synthetic fixtures use regular fan-out and small Markdown records. Results may not generalize to collaborative editing, multilingual notes, very large files, complex YAML, binary-heavy collections, or repositories with millions of nodes. Runtime values are machine-dependent.

Reliability and reproducibility.. The public dataset omits Autodesk text and images to respect third-party rights, which prevents independent semantic evaluation from the archive alone. It does retain URLs, hashes, structural features, and edges. The experiment script, generated fixtures, parameters, raw JSON output, and CSV summaries are public, allowing the reported structural and runtime measurements to be rerun.

Researcher bias.. The system and evaluation were produced by a single independent developer. Fault classes and research questions may reflect the system's design. Public scripts and explicit limitations reduce but do not remove this risk. An independent replication and a field study with multiple maintainers are needed.

9. Conclusion

This study introduced evidence closure as an executable lifecycle invariant for agent-maintained Markdown knowledge. Agent Wiki separates raw evidence from synthesized wiki pages and requires processed sources to declare resolved targets, retain a reciprocal evidence path, and close explicit follow-ups. In a public graph derived from 1,092 technical-documentation sources, all observable closure preconditions had complete coverage. In a controlled 120-fault experiment, evidence closure detected all four fault classes, while an

existence-only checker detected only unresolved targets. End-to-end checking remained below two seconds at the 95th percentile for a 2,080-node fixture on the evaluation machine.

The contribution is not a claim that local Markdown replaces retrieval systems or that structural integrity guarantees truth. It is a smaller systems claim: durable agent knowledge can expose a deterministic, inexpensive, and reviewable path from evidence to synthesis. Future work should evaluate claim-level faithfulness, retrieval quality, maintenance effort, and multi-user operation on independently labeled corpora.

Data and Software Availability

Agent Wiki release `v0.2.1` and the public FlexSim-derived structural dataset are available at https://github.com/NimaChu/agent-wiki and archived at https://doi.org/10.5281/zenodo.21366895. The experiment script and result files accompany this manuscript and will be included in the next archival release before submission. The software is licensed under the MIT License. The original dataset schema, selection, annotations, derived features, and graph structure are licensed under CC BY 4.0. Autodesk-provided documentation text, snapshots, and images are not redistributed.

CRediT Author Statement

Zijun Chu: Conceptualization, methodology, software, validation, investigation, data curation, formal analysis, writing–original draft, writing–review and editing, visualization, and project administration.

Declaration of Competing Interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

During the preparation of this work, the author used OpenAI Codex to assist with manuscript organization, language drafting, LaTeX preparation, and the implementation of reproducible evaluation scripts. The author reviewed and verified the code, data, references, analysis, and manuscript content, edited the material as needed, and takes full responsibility for the content of the article.

Acknowledgements

The author thanks the open-source communities whose tools and documentation support local-first software development and reproducible research.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, International Conference on Learning Representations (2023). https://arxiv.org/abs/2210.03629.

[2] C.E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan, SWE-bench: Can language models resolve real-world GitHub issues?, International Conference on Learning Representations (2024). https://arxiv.org/abs/2310.06770.

[3] J. Yang, C.E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, O. Press, SWE-agent: Agent-computer interfaces enable automated software engineering, Advances in Neural Information Processing Systems 37 (2024). https://arxiv.org/abs/2405.15793.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, Advances in Neural Information Processing Systems 33 (2020) 9459–9474. https://arxiv.org/abs/2005.11401.

[5] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R.O. Ness, J. Larson, From local to global: A graph RAG approach to query-focused summarization, arXiv:2404.16130 (2024). https://arxiv.org/abs/2404.16130.

[6] C. Packer, S. Wooders, K. Lin, V. Fang, S.G. Patil, I. Stoica, J.E. Gonzalez, MemGPT: Towards LLMs as operating systems, arXiv:2310.08560 (2023). https://arxiv.org/abs/2310.08560.

[7] M. Kleppmann, A. Wiggins, P. van Hardenberg, M. McGranaghan, Local-first software: You own your data, in spite of the cloud, in: Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, 2019, pp. 154–178. https://doi.org/10.1145/3359591.3359737.

[8] O.C.Z. Gotel, C.W. Finkelstein, An analysis of the requirements traceability problem, in: Proceedings of the First International Conference on Requirements Engineering, 1994, pp. 94–101. https://doi.org/10.1109/ICRE.1994.292398.

[9] Y. Gil, S. Miles (Eds.), PROV model primer, W3C Working Group Note, World Wide Web Consortium, 2013. https://www.w3.org/TR/prov-primer/.

[10] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J.E.L. Gayo, R. Navigli, S. Neumaier, A.-C.N. Ngomo, A. Polleres, S.M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, A. Zimmermann, Knowledge graphs, ACM Computing Surveys 54(4) (2021) 1–37. https://doi.org/10.1145/3447772.

[11] A. Matuschak, Evergreen notes, 2026. https://notes.andymatuschak.org/ (accessed 15 July 2026).

[12] Obsidian, Obsidian documentation, 2026. https://help.obsidian.md/ (accessed 15 July 2026).

[13] Logseq, Logseq documentation, 2026. https://docs.logseq.com/ (accessed 15 July 2026).